

LES ARBRES BINAIRES DE RECHERCHE

Les arbres binaires de recherche sont des structures de données pouvant supporter bon nombre d'opérations d'ensemble dynamique, notamment RECHERCHER, MINIMUM, MAXIMUM, PRÉDÉCESSEUR, SUCCESSEUR, INSÉRER et SUPPRIMER.

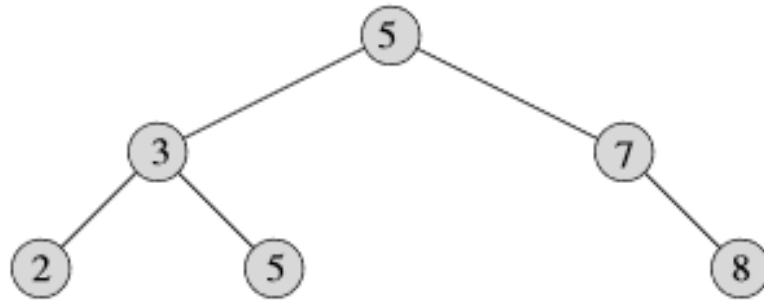
- ❖ Ces opérations s'exécutent généralement en $O(\lg n)$ dans le cas le plus défavorable
- ❖ Chaque nœud contient les champs gauche, droite et p qui pointent sur les nœuds correspondant respectivement à son enfant de gauche, à son enfant de droite et à son parent.

LES ARBRES BINAIRES DE RECHERCHE

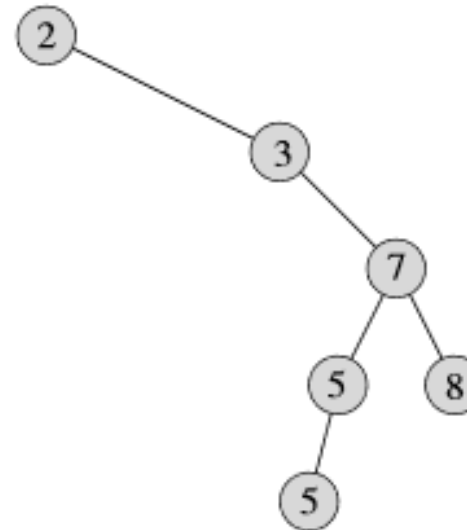
La propriété d'arbre binaire de recherche:

Soit x un nœud d'un arbre binaire de recherche.

- ❖ Si y est un nœud du sous-arbre de gauche de x , alors $y.\text{clé} \leq x.\text{clé}$.
- ❖ Si y est un nœud du sous-arbre de droite de x , alors $y.\text{clé} \geq x.\text{clé}$.



(a)



(b)

LES ARBRES BINAIRES DE RECHERCHE

Parcours infixe → permet d'afficher toutes les clés de l'arbre dans l'ordre trié.
❖ La clé de la racine d'un sous-arbre est imprimée entre les clés du sous-arbre de gauche et les clés du sous-arbre de droite

Parcours-Infixe(x)

Si $x \neq \text{NIL}$ alors

Parcours-Infixe (x.gauche)

afficher x.clé

Parcours-Infixe (x.droite)

Parcours-Préfixe ?

Parcours-Postfixe ?

Arbre-Rechercher-Itératif(x,k)

tant que $x \neq \text{NIL}$ et $k \neq x.\text{clé}$

si $k < x.\text{clé}$

$x = x.\text{gauche}$

sinon

$x = x.\text{droite}$

retourner x

LES ARBRES BINAIRES DE RECHERCHE

Arbre-Minimum(x)
tant que x.gauche $\neq$ NIL
 x=x.gauche
retourner x

Arbre-Maximum(x)
tant que x.droite $\neq$ NIL
 x=x.droite
retourner x

Arbre-Successeur(x)
si x.droite $\neq$ NIL
 retourner Arbre-Minimum(x.droite)
y=x.p
tant que y $\neq$ NIL et x=y.droite
 x=y
 y=y.p
retourner y

LES ARBRES BINAIRES DE RECHERCHE

Procédure insérer dans un
arbre binaire de recherche



Arbre-Insérer(T, z)

```
y=NIL
x=T.racine
tant que x ≠ NIL
  y=x
  si z.clé < x.clé
    x=x.gauche
  sinon
    x=x.droite
y=z.p
si y==NIL
  T.racine =z
sinon si z.clé < y.clé
  y.gauche=z
sinon
  y.droite=z
```

Transplanter(T,u,v)

```
si u.p==NIL
  T.racine=v
sinon si u==u.p.gauche
  u.p.gauche=v
sinon u.p.droite=v
si v ≠ NIL
  u.p=v.p
```

Procédure Supprimer dans un
arbre binaire de recherche



Arbre-Supprimer(T, z)

```
si z.gauche == NIL
  Transplanter(T,z,z.droite)
sinon si z.droite == NIL
  Transplanter(T,z,z.gauche)
Sinon
  y=Arbre-Successeur(z)
  si y.p ≠ z
    Transplanter(T,y,y.droite)
    y.droite=z.droite
    y.droite.p=y
  Transplanter(T,z,y)
  y.gauche=z.gauche
  y.gauche.p=y
```